

GOVERNMENT OF KERALA
DEPARTMENT OF TECHNICAL EDUCATION
RAJIV GANDHI INSTITUTE OF TECHNOLOGY
(GOVT. ENGINEERING COLLEGE)
KOTTAYAM - 686501



MINI PROJECT REPORT

AKSHAYAASSIST

A Digital Service Management and Queue Tracking System

ARYA MANOJ

Master of Computer Applications

Reg. No. KTE25MCA-2013

ABSTRACT

Citizens visiting localized service delivery centers, such as Akshaya centers, often face challenges including long waiting times, manual token management, lack of real-time application tracking, and unstructured document verification processes. Managing service applications manually or through disparate systems makes it difficult for applicants to know the status of their requests and for administrative staff to streamline operations efficiently. To address these limitations, this project proposes **AkshayaAssist**, a web-based digital service management system designed to provide citizens and administrative staff with a centralized platform for managing and monitoring public service applications.

AkshayaAssist allows users to register, securely authenticate, and interact with the system through role-based interfaces tailored for applicants, staff, and administrators. Applicants can submit requests for various public services, upload required supporting documents, and track their application progress in real time through predefined stages such as Submitted, Verification Pending, Approved, and Rejected. The admin and staff modules enable authorized personnel to verify uploaded documents, update application status, manage queue allocations, and generate operational analytics.

The system is built on Django (Python) using the Model-View-Template (MVT) architecture, with a frontend powered by HTML5, CSS3, JavaScript, and Bootstrap. SQLite/PostgreSQL manages data persistence securely. Security is ensured via Role-Based Access Control (RBAC), CSRF protection, and PBKDF2 password hashing. Future enhancements include automated SMS/WhatsApp alerts, payment gateway integration, queue time estimation, and AI document verification.

AkshayaAssist therefore provides a structured, transparent, and efficient approach to public service application handling. By combining a clean front-end presentation with a robust back-end framework, the system aims to minimize counter delays, eliminate manual queue bottlenecks, and significantly improve the citizen service experience.

Keywords: Service Management System, E-Governance, Django, Python, Role-Based Access Control, Web Application, Citizen Services, Public Application Tracking

Contents

1	Introduction	1
1.1	Scope	2
1.2	Relevance	2
1.3	Problem Statement	3
1.4	Objectives	3
1.5	Organisation of the Report	5
2	System Study	6
2.1	Existing System	6
2.1.1	Limitations of the Existing System	6
2.2	Literature Survey	7
2.3	Proposed System	8
2.3.1	Major Functionalities	8
2.4	Development Methodology	9
2.4.1	Sprint-based Development	9
2.5	Requirement Analysis	9
2.5.1	Software Requirements	10
2.5.2	Hardware Requirements	10
2.6	Summary	10
3	Development Methodology	11
3.1	Introduction	11
3.2	Agile Development	11
3.3	Scrum Framework	12
3.4	Product Backlog	12
3.5	Sprint Planning	13
3.6	Sprint Execution	13
3.7	Development Stages	14
3.7.1	Stage 1: Environment and Project Setup	14
3.7.2	Stage 2: Database Modeling	14
3.7.3	Stage 3: Authentication and RBAC Integration	14
3.7.4	Stage 4: Service Catalog and Submission Interface	14
3.7.5	Stage 5: Staff Verification & Processing Desk	14
3.7.6	Stage 6: Queue Management System	15

3.7.7	Stage 7: Testing, Security Hardening, and Polish	15
3.8	Sprint Review	15
3.9	Retrospective and Refinement	15
3.10	Methodology Applied to AkshayaAssist	15
3.11	Advantages of the Selected Methodology	16
3.12	Limitations of the Methodology	16
3.13	Chapter Summary	16
4	Design	17
4.1	System Architecture	17
4.2	Use Case Design	18
4.2.1	Register Account	19
4.2.2	Authenticate / Login	19
4.2.3	Submit Service Application	20
4.2.4	Upload Verification Documents	20
4.2.5	Track Application Status	20
4.2.6	Process Application & Verify Documents	20
4.2.7	Manage Queue Tokens	20
4.2.8	Manage System Services	20
4.3	Data Flow Diagram	20
4.3.1	Level 0 Data Flow Diagram	20
4.3.2	Level 1 Data Flow Diagram	21
4.4	Activity Design	22
4.4.1	User Authentication Activity	22
4.4.2	Application Submission & Processing Activity	22
4.5	Database Design	23
4.6	User Interface Design	26
4.7	Security Design	26
4.8	Design Summary	26
5	Implementation	27
5.1	Development Environment	27
5.2	Application Structure	27
5.3	Application Routing	29
5.4	Authentication and Role Control Implementation	29
5.5	Service Application and Document Upload	30
5.6	Staff Verification Desk & Processing	30
5.7	Queue Token Management System	30

5.8	Database Implementation	31
5.9	Security Implementation	31
5.10	Testing of Implemented Features	31
5.11	Summary	32

List of Figures

4.1	System Architecture of AkshayaAssist	18
4.2	Use Case Diagram of AkshayaAssist	19
4.3	Level 0 Data Flow Diagram	21
4.4	Level 1 Data Flow Diagram	21
4.5	User Authentication Activity Diagram	22
4.6	Application Processing Activity Diagram	23
4.7	Relational Database Entity Relationship Diagram	25

List of Tables

2.1	Software Requirements	10
2.2	Hardware Requirements	10
4.1	Core Database Schema Entities	24
5.1	Application Routes	29
5.2	Functional Test Cases	32

Introduction

Public service delivery systems at the grassroots level, such as Akshaya e-Centres in Kerala, play a crucial role in delivering government-to-citizen (G2C) and business-to-citizen (B2C) services. Citizens frequently visit these centers for various administrative requirements, including identity registrations, certificates, welfare scheme applications, and utility bill payments. However, managing these daily interactions, applicant queues, and service requests using manual registers or fragmented tools often leads to long waiting times, lack of real-time application tracking, and operational bottlenecks for center administrators.

At present, applicants and center operators rely heavily on manual token distribution, physical counter visits, and paper-based document verification. While centralized state portals handle major backend processing, applicants lack a direct, localized mechanism to monitor their application stages, estimate waiting times, or pre-verify their uploaded documentation prior to visiting the center. This causes unnecessary crowd gathering at counters and increases the workload on center staff for simple status queries.

AkshayaAssist is proposed as a comprehensive web-based service management and queue tracking platform designed to bridge this operational gap. The system provides applicants, office staff, and system administrators with a centralized digital workspace. Applicants can securely log in, submit service requests, upload supporting documents, and monitor the real-time status of their applications. Simultaneously, staff and administrators can manage citizen queues, verify uploaded documents, update processing stages, and review center operations.

The system is developed using robust, modern web architecture principles. The backend service and business logic are implemented using **Python** and the **Django Web Framework**. The frontend user interface is built using **HTML5**, **CSS3**, **JavaScript**, and **Bootstrap** to ensure responsiveness across desktop and mobile devices. Database management for users, service applications, queues, and document records is handled via **SQLite/PostgreSQL**.

The initial implementation focuses on core digital center operations, including user registration and authentication, role-based access control (RBAC), application submission, queue token management, and administrative verification workflows. The modular architecture also provides scope for extending the system with advanced features such as automated notifications, payment gateway integration, dynamic waiting time estimation, and AI-assisted document validation.

1.1 Scope

The scope of AkshayaAssist is to establish a streamlined, digitalized environment for managing localized public service applications and counter operations. The project aims to eliminate reliance on manual paperwork and physical queues, offering a structured workflow for applicants and service providers alike.

The primary scope includes secure multi-user registration and Role-Based Access Control (RBAC). Three distinct user roles are supported: Applicants (citizens), Center Staff, and Administrators. Authentication mechanisms ensure that sensitive personal documents and application details are securely segregated and accessible only to authorized entities.

Application lifecycle management forms the core functionality of AkshayaAssist. Citizens can browse available services, submit digital application forms, upload required supporting files, and track progress through defined stages such as *Submitted*, *Verification Pending*, *Approved*, or *Rejected*.

The dashboard interface provides customized views based on user roles. Applicants receive a unified view of their pending and completed applications, while center staff receive an operational desk to process requests, manage queue numbers, and verify uploaded documentation.

The architecture allows for scalable expansion. Future scope includes integrating SMS/WhatsApp notification channels, online fee payment handling, automated analytics for center performance, and machine learning models for document automated checks.

1.2 Relevance

The relevance of AkshayaAssist stems from the urgent need for digital transformation in local administrative service delivery. Physical congestion at public service counters, administrative overhead, and opaque application progress tracking cause frustration for citizens and inefficiency for staff.

A dedicated local service management web application resolves these issues by enabling pre-application submission, digital document verification, and queue optimization. Instead of standing in queues merely to inquire about status or submit basic details, citizens can interact with the center digitally.

From a software engineering perspective, AkshayaAssist demonstrates the application of full-stack web engineering concepts using Python and Django. It incorporates Model-View-Template (MVT) design patterns, relational database modeling, dynamic template rendering, session management, security best practices (CSRF, password hashing), and

user-centric frontend design.

Furthermore, the system lays the foundation for smart governance by providing structured operational data that can be analyzed to evaluate peak visitor hours, processing efficiency, and service demand patterns.

1.3 Problem Statement

Citizens visiting service centers frequently face significant delays due to manual registration, unstructured queue handling, and lack of clarity regarding required document submission. Additionally, tracking the progress of submitted applications often requires physical visits to the center, leading to counter congestion.

On the administrative side, staff members must manually review paper documents, manage physical tokens, and update offline records, which increases the likelihood of human error and operational slowdowns.

The problem addressed by this project is the lack of a unified, web-based digital management system that streamlines application processing, document verification, and queue management for local service centers. AkshayaAssist addresses this challenge by providing a web platform combining secure authentication, role-based workflows, dynamic application tracking, and structured database storage.

1.4 Objectives

The primary objective of AkshayaAssist is to build a web-based public service management and queue tracking platform that enhances operational efficiency and citizen convenience.

The specific objectives include:

1. To design and develop a responsive user interface using Bootstrap, HTML, CSS, and JavaScript.
2. To implement secure authentication and Role-Based Access Control (RBAC) using Django's authentication system.
3. To enable citizens to submit applications online along with digital document uploads.
4. To provide a real-time application tracking mechanism with clear status indicators.
5. To build an administrative interface for staff to inspect documents, update processing status, and manage queue numbers.

6. To store and manage application, user, and service records securely using a relational database (SQLite/PostgreSQL).
7. To maintain a modular web application architecture adhering to the Django Model-View-Template (MVT) design pattern.
8. To establish a scalable framework capable of integrating payment handling, automated notifications, and analytical reporting in future iterations.

1.5 Organisation of the Report

This report is organized into five chapters. Chapter 1 introduces AkshayaAssist, outlining its scope, relevance, problem statement, objectives, and report structure.

Chapter 2 presents the system study. It details existing public service workflows, the proposed solution, literature survey, and functional and technical requirements.

Chapter 3 describes the development methodology, covering Agile Scrum practices, sprint planning, execution, and review activities.

Chapter 4 details the system design, including software architecture, database schema, entity-relationship models, use cases, dynamic data flow diagrams, and user interface design.

Chapter 5 presents the implementation details, discussing the development setup, Django backend configurations, database modeling, frontend template integration, testing methodologies, and deployment preparation.

System Study

The system study phase involves analyzing traditional public service counter operations, identifying bottleneck procedures in application handling, and evaluating technological solutions to streamline workflow management. This study establishes the theoretical foundation for AkshayaAssist, justifying technology choices and defining clear requirements.

AkshayaAssist addresses operational delays, lack of status visibility, and manual paper handling in local public service centers. The system study reviews current service workflows, literature in e-governance, proposed architectural solutions, and essential hardware/software environments.

2.1 Existing System

Currently, citizens seeking services at local centers must physically visit counters to collect application forms, submit paper documents, obtain queue tokens, and inquire about application statuses. Center staff manually verify physical copies, enter details into legacy systems or registers, and manually manage physical crowds.

Although individual high-level government portals exist for specific statutory certificates, localized interaction, initial submission help, digital token management, and preliminary document verification are still largely managed offline or through fragmented procedures at individual centers.

This manual process forces applicants to make multiple physical visits for single service requests—first to submit, then to correct document errors, and finally to collect outputs.

The existing workflow operates as a collection of manual, paper-reliant counter steps rather than a centralized, digitalized local service environment.

2.1.1 Limitations of the Existing System

The major limitations identified in the existing system are:

1. **High Counter Congestion:** Lack of online queuing or pre-application registration results in long physical waiting queues at centers.
2. **Manual Document Verification:** Physical paper checking at counters increases processing time per applicant.

3. **Lack of Real-Time Tracking:** Applicants cannot view application status online and must visit the center for status inquiries.
4. **Risk of Document Errors:** Incomplete or incorrect paper document submissions are often identified late in the process.
5. **High Administrative Overhead:** Center staff spend substantial effort on manual token distribution and repeated query handling.
6. **Limited Extensibility:** Manual records cannot easily be integrated with automated notifications, digital queue displays, or operational analytics.

2.2 Literature Survey

Digital governance and citizen service automation have been widely studied to evaluate how web frameworks, queue algorithms, and role-based management portals improve service efficiency.

Existing literature on e-governance platforms emphasizes that web-based self-service portals significantly reduce counter processing times and operational costs. Systems integrating pre-verification workflows allow staff to approve or request document corrections prior to physical appointments, optimizing in-person queue management.

Research in public administrative automation highlights the importance of Role-Based Access Control (RBAC). Systems serving multiple user categories (e.g., applicants, counter staff, system admins) require strict data isolation and permission levels to protect personal data while maintaining operational integrity.

From a software engineering perspective, modern Model-View-Template (MVT) frameworks like Django provide built-in security mechanisms (such as protection against SQL injection, Cross-Site Scripting, and Cross-Site Request Forgery) along with Object-Relational Mapping (ORM) capabilities that accelerate secure web development compared to traditional monolithic scripts.

The literature survey highlights the following core design considerations for AkshayaAssist:

- Centralized web platform with role-based access for Citizens, Staff, and Admins.
- Structured database schema to handle users, services, applications, and documents.
- Clear status workflow for application lifecycle tracking.
- Clean, responsive UI for easy access on mobile and desktop browsers.
- Scalable modular architecture using Django for future extension.

2.3 Proposed System

AkshayaAssist is designed as a centralized digital service and queue management web application. It provides a secure, role-based platform for citizens to apply for services, submit documents, and track progress, while providing staff with tools to review applications and manage service queues efficiently.

The application allows citizens to log in, view available services, upload required documents, and receive unique application reference numbers. Center staff use a dedicated portal to view incoming requests, review uploaded attachments, mark applications as approved/rejected/pending-action, and manage daily service tokens.

The system is constructed using Django (Python) on the backend and HTML5, CSS3, JavaScript, and Bootstrap on the frontend, with SQLite/PostgreSQL as the relational database engine.

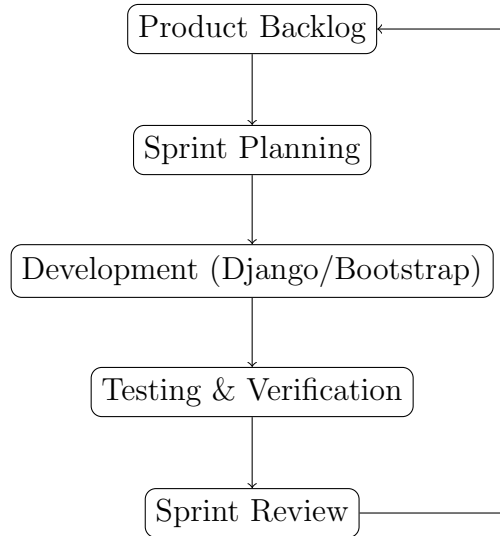
2.3.1 Major Functionalities

The key functionalities of AkshayaAssist include:

1. **User Authentication & Authorization:** Secure login and signup with dynamic redirection based on user roles (Applicant, Staff, Admin).
2. **Service Catalog Management:** Admins can add, update, or configure available public services and required document lists.
3. **Application Submission & Uploads:** Applicants can fill out digital service forms and upload supporting documentation.
4. **Real-Time Status Tracking:** Live status updates (*Submitted, Under Review, Approved, Rejected*) visible on applicant dashboards.
5. **Staff Processing Portal:** Interface for center staff to view applications, verify uploaded documents, and update statuses.
6. **Queue Management System:** Digital token generation and allocation to minimize counter rush.
7. **Relational Database Persistence:** Secure relational data handling using Django ORM backed by SQLite/PostgreSQL.

2.4 Development Methodology

AkshayaAssist was developed following an Agile methodology with a Scrum framework. Development was organized into iterative sprints, allowing functionality to be built, tested, and refined incrementally.



2.4.1 Sprint-based Development

The project was executed in sequential sprints:

- **Sprint 1:** Requirements gathering, database schema modeling, and base Django environment setup.
- **Sprint 2:** Authentication modules, user role configuration, and Bootstrap UI layout integration.
- **Sprint 3:** Service catalog implementation, application submission workflows, and file upload handling.
- **Sprint 4:** Staff verification interface, queue management logic, dashboard integration, and comprehensive testing.

2.5 Requirement Analysis

The system requirements are categorized into software and hardware specifications necessary for developing and executing AkshayaAssist.

2.5.1 Software Requirements

Table 2.1: Software Requirements

Software / Technology	Purpose
Python 3.x	Core programming language for backend logic.
Django Framework	Web application framework (MVT architecture).
HTML5 / CSS3	Structuring and styling frontend templates.
JavaScript	Client-side form validation and dynamic UX.
Bootstrap 5	Responsive CSS framework for UI design.
SQLite / PostgreSQL	Relational database management system.
Git & GitHub	Version control and repository management.
Web Browser	Application execution and interface testing.

2.5.2 Hardware Requirements

Table 2.2: Hardware Requirements

Component	Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or higher.
RAM	Minimum 4 GB (8 GB recommended).
Storage	5 GB available disk space.
Display	Standard HD Display (1280 × 720 resolution or higher).
Network	Internet connection for dependency installation and web deployment.

2.6 Summary

The system study identified key operational bottlenecks in manual service center handling and established requirements for a web-based solution. AkshayaAssist addresses these issues through an integrated Django platform supporting role-based authentication, online application processing, digital document validation, and queue tracking. The specifications defined in this chapter form the foundation for the architectural designs detailed in subsequent chapters.

Development Methodology

3.1 Introduction

The development methodology defines the approach followed for planning, developing, testing, and refining the AkshayaAssist service and queue tracking system. Since administrative workflows and service specifications can evolve during development, an iterative methodology was selected instead of a strictly sequential model.

AkshayaAssist was developed using an Agile approach with Scrum-inspired practices. The project was divided into smaller development tasks, with each sprint focusing on a specific set of features. Functionality was implemented incrementally and tested throughout the development lifecycle.

The methodology provided a structured framework for managing the project while allowing implementation challenges and architectural decisions to be addressed dynamically.

3.2 Agile Development

Agile software development is an iterative and incremental approach where software evolves through repeated cycles of planning, implementation, testing, and refinement. Instead of attempting to build the entire system in a single phase, the overall application is divided into functional components.

The Agile approach is particularly suitable for systems where requirements may be refined during development. It allows continuous feedback during testing to improve subsequent development cycles.

For AkshayaAssist, an iterative approach was essential because several modules depended heavily on one another. For example, user authentication and Role-Based Access Control (RBAC) had to be established before staff and applicant dashboards could be implemented, while database interactions through Django ORM depended on model definitions.

The core principles applied during development were:

- Incremental building of system functionality.
- Continuous testing during template rendering and view logic execution.
- Early identification and resolution of bugs.

- Division of work into manageable tasks.
- Continuous refinement based on testing results.

3.3 Scrum Framework

Scrum is an Agile framework that organizes development into short, iterative cycles known as sprints. Each sprint focuses on completing a specific set of prioritized features.

The core Scrum concepts applied to AkshayaAssist include:

- **Product Backlog:** A prioritized list of technical and functional requirements.
- **Sprint:** A fixed-length development cycle dedicated to specific backlog items.
- **Sprint Planning:** Identifying tasks to be executed in an upcoming iteration.
- **Sprint Review:** Evaluating completed features against requirements.
- **Retrospective:** Analyzing challenges to improve subsequent development sprints.

3.4 Product Backlog

The product backlog for AkshayaAssist was created by identifying functional requirements for applicants, staff, and administrators.

The initial backlog included:

1. Environment setup (Python, Django, Bootstrap integration).
2. Database schema design (Users, Services, Applications, Queue Tokens).
3. Custom user authentication and RBAC implementation.
4. Applicant registration and login views.
5. Service catalog display and management.
6. Application submission form with document file upload.
7. Real-time application tracking dashboard for applicants.
8. Staff verification dashboard for document inspection and status updates.
9. Digital queue token generation logic.
10. Admin portal for system and service configuration.

11. Security configurations (CSRF protection, media file access rules).
12. Integration and user acceptance testing.

3.5 Sprint Planning

Sprint planning involved ordering tasks by dependencies. Base project configuration and authentication models were established first, followed by application submission workflows, staff verification portals, and queue management modules.

The development progression is illustrated below:

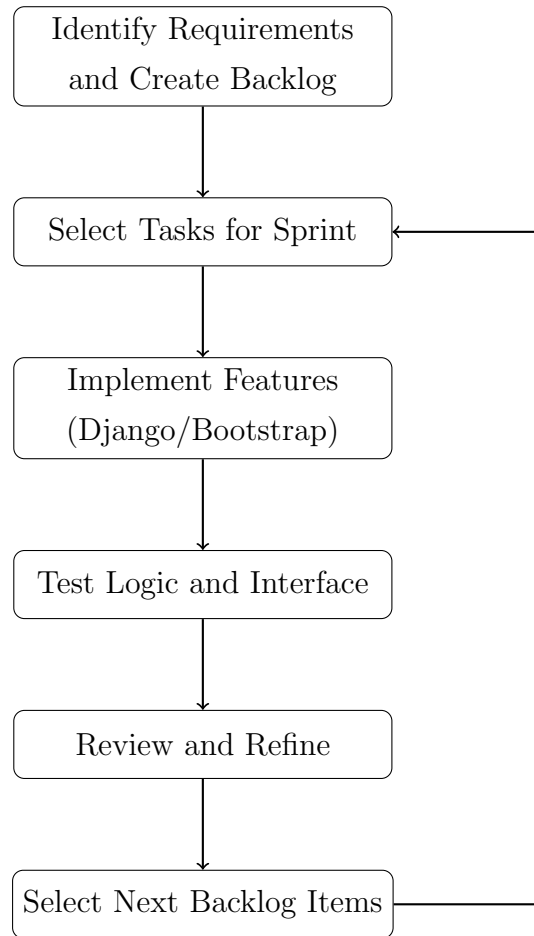


Figure 3.1: Iterative Development Process

3.6 Sprint Execution

During sprint execution, selected tasks were implemented incrementally using Django's Model-View-Template (MVT) architecture.

Initial activities focused on creating the Django project, establishing virtual environments, configuring database connections, and designing base HTML templates using Bootstrap.

Next, authentication modules were implemented. Django’s built-in user authentication was extended to support role-based privileges (Applicant, Staff, Admin). Session security and view decorators (e.g., `@login.required`) were implemented to safeguard sensitive pages.

Once authentication was validated, the application submission module was constructed. Models for service requests and uploaded files were created, and HTML forms were built to support secure file uploads.

Finally, staff processing desks and queue management features were integrated. Views were written to allow staff to update application statuses dynamically and issue digital tokens.

3.7 Development Stages

The development of AkshayaAssist was conducted in seven major stages:

3.7.1 Stage 1: Environment and Project Setup

Configured the Python environment, initialized the Django application, structured static assets (CSS/JS/images), and established base layout templates using Bootstrap.

3.7.2 Stage 2: Database Modeling

Defined relational data models in `models.py` for Users, Roles, Services, Applications, Documents, and Queue Tokens, followed by running initial database migrations.

3.7.3 Stage 3: Authentication and RBAC Integration

Implemented secure signup, login, and logout functionalities. Built Role-Based Access Control logic to restrict access based on user authorization levels.

3.7.4 Stage 4: Service Catalog and Submission Interface

Created dynamic views and templates for applicants to browse services, fill out application forms, and upload required verification documents.

3.7.5 Stage 5: Staff Verification & Processing Desk

Built administrative views enabling staff to inspect submitted documents, update status indicators (*Pending*, *Approved*, *Rejected*), and add notes.

3.7.6 Stage 6: Queue Management System

Engineered the token generation logic to assign queue numbers to users upon physical or digital check-in, updating real-time counter displays.

3.7.7 Stage 7: Testing, Security Hardening, and Polish

Executed functional, regression, and security testing. Configured CSRF token validation, input sanitization, and media upload protections.

3.8 Sprint Review

At the end of each development cycle, features were evaluated against functional requirements. Views, template renderings, form submissions, and database state updates were checked thoroughly.

For example, after developing the document submission module, file uploads were tested with various file formats and sizes to ensure correct handling by Django's storage handlers.

3.9 Retrospective and Refinement

Retrospective evaluations helped address implementation roadblocks. For instance, during media file upload testing, initial permissions restricted staff from viewing submitted files. This was resolved by refining media URL routes and adjusting staff file view access permissions within Django.

3.10 Methodology Applied to AkshayaAssist

The progression of development activities for AkshayaAssist followed these structured steps:

1. Define domain requirements for public service management.
2. Initialize Django project structure and static configurations.
3. Model relational schemas using Django ORM.
4. Implement user authentication and role differentiation.
5. Construct applicant submission interfaces and document upload handlers.
6. Build staff processing portals and queue management components.

7. Secure routes using role-based decorators.
8. Perform end-to-end integration and user acceptance testing.

3.11 Advantages of the Selected Methodology

- **Incremental Development:** Core modules were delivered and verified independently.
- **Early Error Detection:** Issues in data modeling or view logic were identified early.
- **Flexibility:** Adaptable to adjustments in service workflow logic.
- **Risk Reduction:** System architecture stability was verified step-by-step.

3.12 Limitations of the Methodology

In an academic project context, full Scrum ceremonies (such as daily standups with distinct Product Owners and Scrum Masters) were condensed into individual engineering workflows. However, the core principles of iterative development and continuous verification were fully maintained.

3.13 Chapter Summary

This chapter detailed the Agile Scrum methodology used to develop AkshayaAssist. The iterative approach supported effective planning, modular implementation using Django and Bootstrap, continuous integration, and systematic testing.

Design

The design phase translates functional requirements into a detailed technical blueprint for AkshayaAssist. It defines the system architecture, component interactions, data flow, relational database design, and user interface structure. The modular architecture ensures maintainability and scalability for future enhancements.

AkshayaAssist follows the Model-View-Template (MVT) architectural pattern provided by Django. The presentation layer uses HTML5, CSS3, JavaScript, and Bootstrap, while business logic and database management are handled through Python and SQLite/PostgreSQL.

4.1 System Architecture

The system architecture describes the structural components of AkshayaAssist and their interconnections. The architecture is organized into four distinct layers: Presentation, Routing/Control, Business Logic, and Data Persistence.

The presentation layer provides intuitive interfaces for Applicants, Staff, and Administrators. Bootstrap ensures responsive design across mobile devices and desktop workstations.

The routing and control layer uses Django's URL dispatcher to route requests to appropriate view handlers. Protected routes enforce authentication and role checks.

The business logic layer contains view controllers and form processing modules that handle application submissions, status updates, document validations, and queue calculations.

The data persistence layer uses Django ORM to interact securely with the relational database (SQLite during development, PostgreSQL for production), managing user data, application states, and media records.

The overall architecture is shown in Figure 4.1.

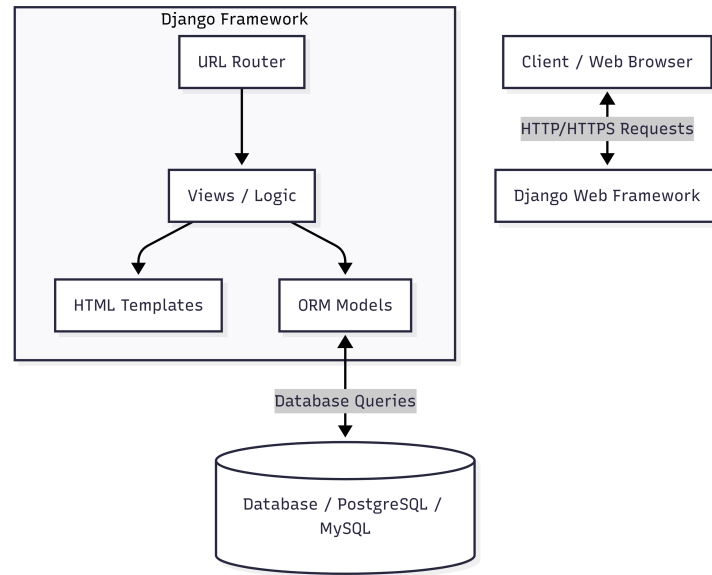


Figure 4.1: System Architecture of AkshayaAssist

The primary architectural layers include:

1. **Presentation Layer:** HTML5, CSS3, JavaScript, and Bootstrap interface components.
2. **Routing & Control Layer:** Django URL routing engine and access control decorators.
3. **Business Logic Layer:** Django Views, forms, file handling modules, and queue processing logic.
4. **Data Layer:** Relational database managed through Django Object-Relational Mapping (ORM).

4.2 Use Case Design

Use case modeling outlines interactions between primary actors and AkshayaAssist. The system identifies three main actors: **Applicant** (Citizen), **Center Staff**, and **Administrator**.

Key use cases include:

- Register Account
- Authenticate / Login
- Browse Services

- Submit Service Application
- Upload Verification Documents
- Track Application Status
- Process Application (Staff)
- Verify Uploaded Documents (Staff)
- Manage Queue Tokens (Staff/Admin)
- Manage System Services (Admin)
- Logout

The Use Case Diagram is shown in Figure 4.2.

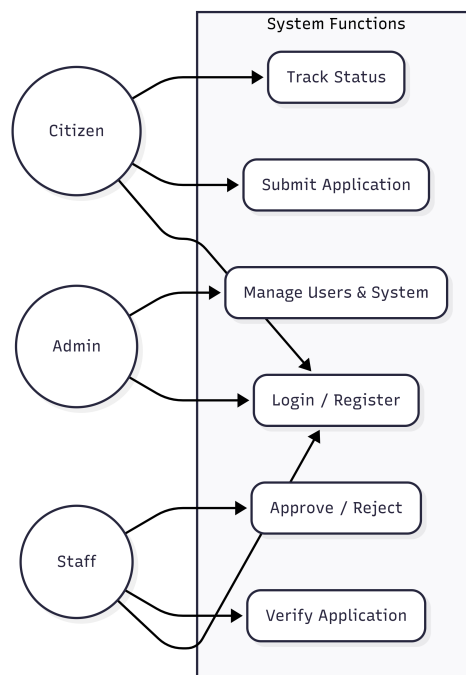


Figure 4.2: Use Case Diagram of AkshayaAssist

4.2.1 Register Account

Allows citizens to create an applicant account by providing contact details and credentials.

4.2.2 Authenticate / Login

Validates user credentials and directs users to their respective role-specific dashboard (Applicant Desk, Staff Portal, or Admin Panel).

4.2.3 Submit Service Application

Enables applicants to select a public service, fill in required fields, and submit an application.

4.2.4 Upload Verification Documents

Allows applicants to upload digital copies of supporting identity and address documents.

4.2.5 Track Application Status

Provides applicants with real-time updates on their application processing state (*Submitted, Verification Pending, Approved, Rejected*).

4.2.6 Process Application & Verify Documents

Allows center staff to open pending applications, inspect uploaded document files, update processing stages, and append administrative notes.

4.2.7 Manage Queue Tokens

Generates and manages counter token numbers for physical center visitors to organize counter workflow.

4.2.8 Manage System Services

Enables administrators to add, update, or disable public service offerings and configure document requirements.

4.3 Data Flow Diagram

Data Flow Diagrams (DFDs) illustrate how information moves through AkshayaAssist between external entities, system processes, and data stores.

4.3.1 Level 0 Data Flow Diagram

The Context-Level DFD depicts the overall system interaction between users (Applicants, Staff, Admins) and the AkshayaAssist platform.

The Level 0 DFD is shown in Figure 4.3.

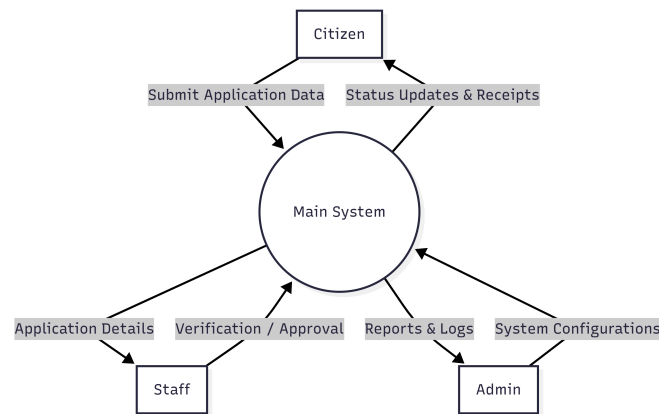


Figure 4.3: Level 0 Data Flow Diagram

4.3.2 Level 1 Data Flow Diagram

The Level 1 DFD decomposes the core platform into detailed sub-processes: Authentication, Service Management, Application Submission, Document Validation, and Queue Tracking.

The Level 1 DFD is shown in Figure 4.4.

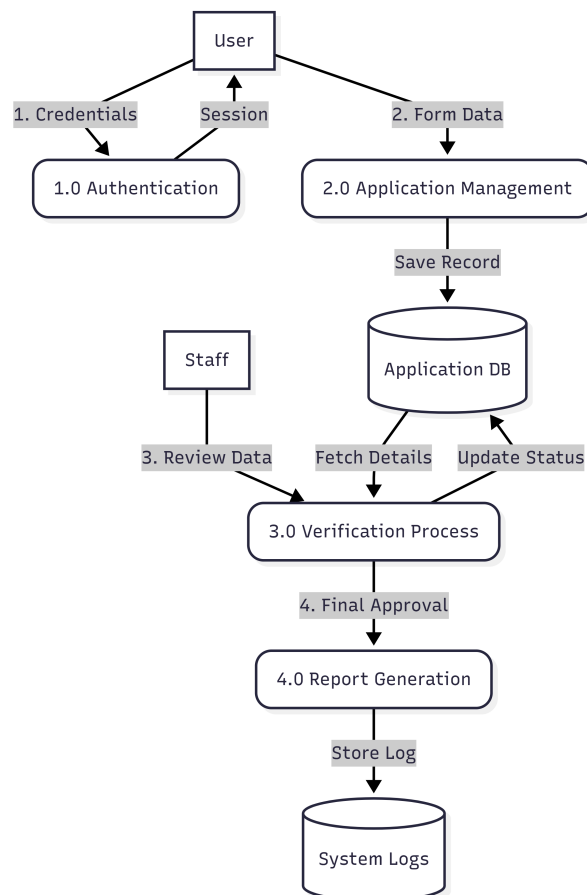


Figure 4.4: Level 1 Data Flow Diagram

4.4 Activity Design

Activity diagrams model workflow logic and decision processes within the system.

4.4.1 User Authentication Activity

Illustrates user credential validation, role checking, and redirection to the appropriate portal.

The activity flow is shown in Figure 4.5.

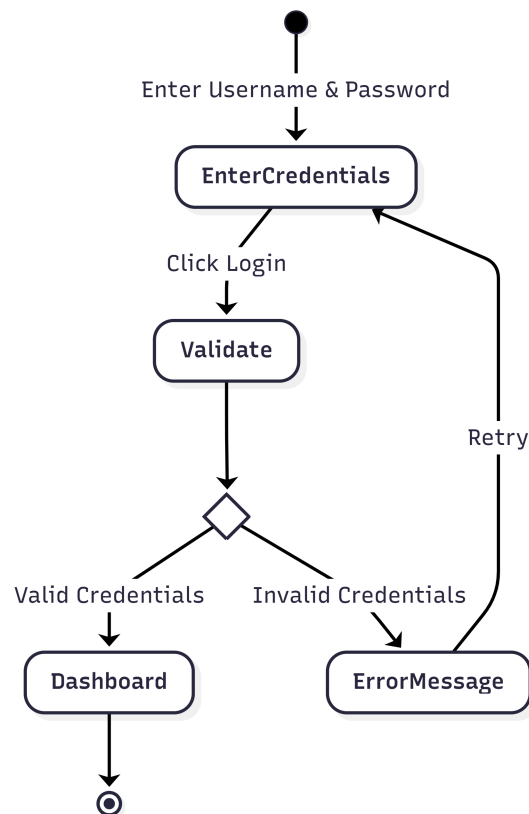


Figure 4.5: User Authentication Activity Diagram

4.4.2 Application Submission & Processing Activity

Models the workflow from application submission by an applicant to document review and status update by staff.

The activity flow is shown in Figure 4.6.

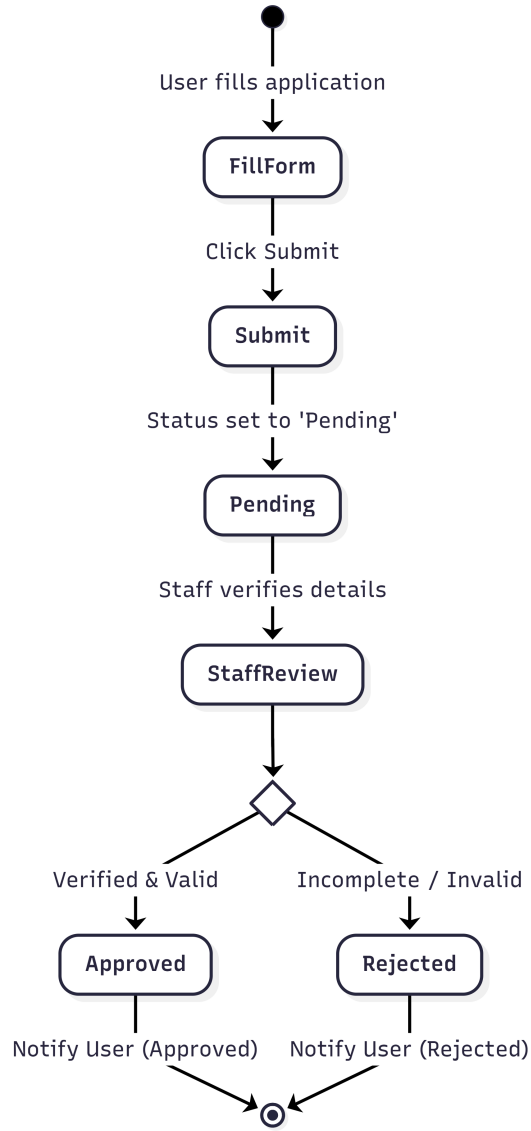


Figure 4.6: Application Processing Activity Diagram

4.5 Database Design

AkshayaAssist uses a relational database structure managed through Django ORM. The relational model ensures data integrity, enforces foreign key relationships, and supports secure querying.

The database entity structures are summarized in Table 4.1.

Table 4.1: Core Database Schema Entities

Entity / Field	Description
User (Custom User)	Stores account details, hashed passwords, and user role identifiers (<i>Applicant</i> , <i>Staff</i> , <i>Admin</i>).
Service	Defines available public services, descriptions, processing fees, and required document check-lists.
Application	Holds application entries linked to a User and Service, containing reference codes and status flags.
Document	Stores media file paths, document types, and verification status linked to an Application.
QueueToken	Stores generated queue token numbers, counter assignments, timestamp, and active queue status.

The database relational structure is illustrated in Figure 4.7.

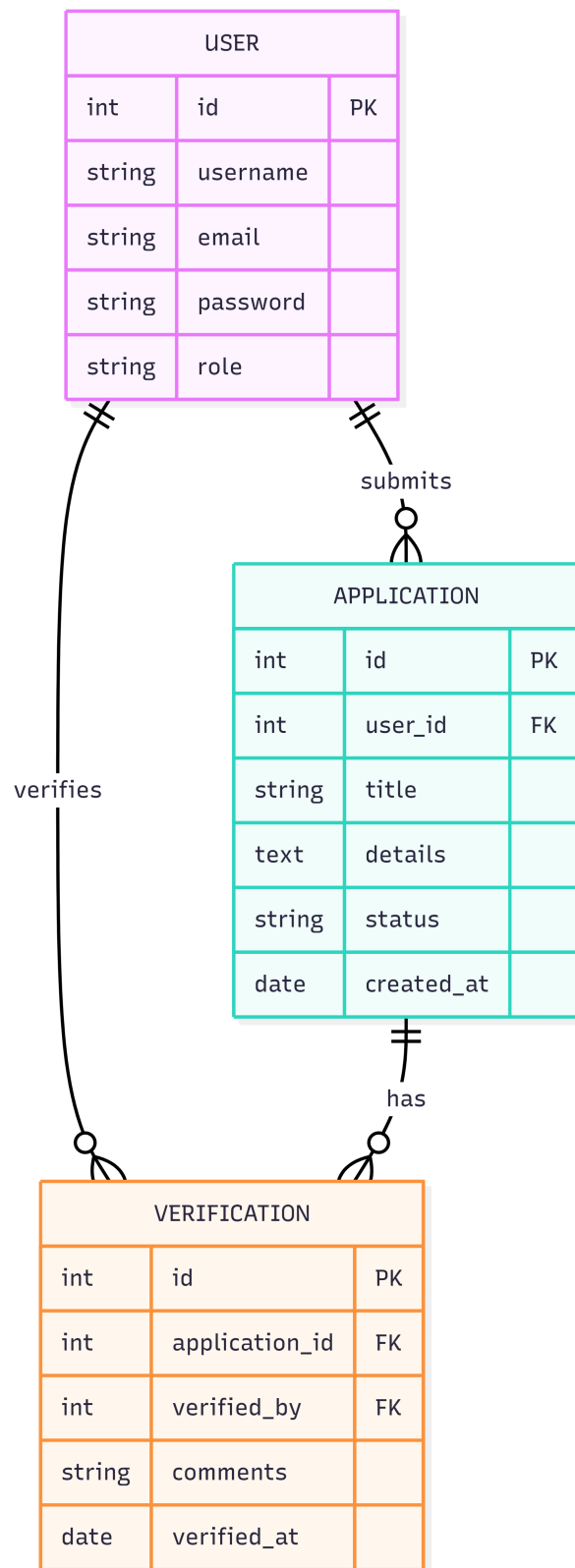


Figure 4.7: Relational Database Entity Relationship Diagram

4.6 User Interface Design

The frontend interface is designed for clarity, accessibility, and responsiveness using Bootstrap components.

Key interfaces include:

- **Home Page:** Introduction to available services and quick access to login/registration portals.
- **Authentication Pages:** Clean login and registration views with form validation styling.
- **Applicant Dashboard:** Central dashboard showing active applications, real-time status badges, and token details.
- **Application Submission Form:** Structured form with file inputs for document attachments.
- **Staff Verification Portal:** Desk interface for reviewing pending applications and inspecting uploaded files.

4.7 Security Design

Security mechanisms integrated into AkshayaAssist include:

- **Authentication & RBAC:** Django's built-in session framework ensures users access only authorized views.
- **CSRF Protection:** All POST forms include Django CSRF tokens to prevent cross-site request forgery attacks.
- **SQL Injection Prevention:** Django ORM uses parameterized queries, neutralizing SQL injection vectors.
- **Secure Media Access:** Uploaded documents are isolated with restricted view permissions to prevent unauthorized access.

4.8 Design Summary

The design of AkshayaAssist provides a complete technical blueprint covering architecture, use cases, data flow, relational database design, user interfaces, and security configurations. The modular Django MVT design ensures clear separation of concerns, providing a strong foundation for implementation details discussed in the next chapter.

Implementation

The implementation phase converts the design specifications of AkshayaAssist into a fully functional, web-based service management and queue tracking platform. The application is built using Django's Model-View-Template (MVT) architecture, Python backend logic, Bootstrap UI components, and a relational database (SQLite/PostgreSQL).

The implementation focuses on core operational features: user registration, role-based authentication, application submission with document uploads, staff verification desks, queue token generation, and real-time status tracking dashboards.

5.1 Development Environment

AkshayaAssist is constructed using a robust Python web development setup. The backend utilizes the Django Web Framework, while the frontend is crafted using standard web technologies enhanced with Bootstrap for responsive styling.

The primary stack elements used in the implementation are:

- **Python 3.x:** Core programming language for server-side execution.
- **Django Web Framework:** Full-stack Python framework providing ORM, authentication, and templating engines.
- **HTML5 & CSS3:** Core structure and visual presentation layer.
- **JavaScript:** Handles asynchronous form submission and dynamic UI element behaviors.
- **Bootstrap 5:** Responsive framework ensuring desktop and mobile compatibility.
- **SQLite / PostgreSQL:** Relational database management system for persistent storage.
- **Git & GitHub:** Version management and source code repository hosting.

5.2 Application Structure

The application architecture follows Django's modular app structure. Business logic, data models, views, and templates are structured inside dedicated modules within the main project.

The core pages implemented include:

- Home / Portal Overview
- User Authentication (Login, Applicant Registration)
- Citizen Application Portal (Service Selection, Application Form, File Uploads)
- Applicant Dashboard (Real-time Status Tracking, Token Details)
- Staff Verification Desk (Application Review, Document Inspection, Status Actions)
- Queue Management Desk (Token Generation and Counter Display)
- Admin Panel (Service Catalog Management, User Role Configuration)

A simplified representation of the project workspace is presented below:

```
akshaya_assist/
|
+-- akshaya_assist/          # Project Core Configuration
|   +-- settings.py         # App settings, Database & Media configurations
|   +-- urls.py             # Root URL router
|   +-- wsgi.py             # Application server deployment interface
|
+-- services/               # Main Application Module
|   +-- models.py           # Custom User, Service, Application, Document & Queue mo
|   +-- views.py            # Business logic for Applicants, Staff, and Admins
|   +-- forms.py            # Django forms with file upload handlers
|   +-- urls.py             # App-level routing paths
|   +-- admin.py            # Admin panel customizations
|
+-- templates/              # HTML Templates
|   +-- base.html           # Core layout wrapper with Bootstrap navbar
|   +-- home.html           # Landing page template
|   +-- login.html          # Authentication views
|   +-- register.html       # Citizen registration form
|   +-- dashboard.html      # Applicant status desk
|   +-- apply_service.html   # Service application & document upload form
|   +-- staff_portal.html    # Staff document review interface
|
+-- static/                 # Static Assets (Custom CSS, JS, Images)
```

```

+-- media/                # Uploaded Citizen Verification Documents
+-- manage.py             # Django CLI execution tool

```

5.3 Application Routing

Django’s URL dispatcher maps incoming browser requests to corresponding view functions in `views.py`. The key routing endpoints are detailed in Table 5.1.

Table 5.1: Application Routes

Route	Purpose
/	Home / Landing Page
/login/	User login authentication entry point
/register/	New applicant account creation
/dashboard/	Applicant real-time status portal
/services/	Service catalog browser
/apply/{int:service_id}/	Application submission form with document upload
/staff/portal/	Staff application verification & queue management desk
/admin/	Administrative management panel

Access control mechanisms ensure that unauthorized users attempting to open restricted endpoints (e.g., `/staff/portal/`) are redirected to authentication pages.

5.4 Authentication and Role Control Implementation

Authentication is managed via Django’s authentication system, customized to support Role-Based Access Control (RBAC). Three operational user roles are defined: *Applicant*, *Staff*, and *Admin*.

The authentication process works as follows:

1. The user submits credentials via the login form.
2. Authentication views execute Django’s `authenticate()` engine.
3. Upon credential verification, user role attributes are checked.
4. The session is created, and the user is redirected to their role-specific desk (`/dashboard/` for applicants, `/staff/portal/` for staff).

View security decorators such as `@login_required` and custom role checks prevent illegal access across user privilege tiers.

5.5 Service Application and Document Upload

The application submission pipeline allows citizens to select a public service, input required details, and upload supporting documents.

Key upload features include:

- Structured entry fields for personal details and service requirements.
- Dynamic file fields allowing upload of verification files (PDFs/Images).
- Back-end file sanitization and attachment linking to specific application instances.
- Automated generation of unique application tracking reference codes.

5.6 Staff Verification Desk & Processing

Center staff utilize a dynamic verification portal to manage incoming applications.

The verification process includes:

1. Staff members view pending service requests in a tabular format.
2. Selecting an application opens detailed applicant information alongside uploaded media documents.
3. Staff review the uploaded files for validity and completeness.
4. The application state is updated to *Approved*, *Rejected*, or *Action Required*, accompanied by optional notes.
5. State updates reflect instantly on the citizen's personal dashboard.

5.7 Queue Token Management System

To streamline counter visits, AkshayaAssist features an automated queue token management workflow.

When an applicant visits the center or checks in online:

1. The system generates a daily sequential token number (e.g., T-015).
2. Active token numbers, estimated wait times, and serving counters are displayed on the public queue status board.

3. Staff update counter numbers as service requests complete, advancing the queue sequence dynamically.

5.8 Database Implementation

Data persistence is implemented using Django ORM, interacting with an underlying relational database engine.

The core database schemas managed in `models.py` are:

```
CustomUser (AbstractUser)
|
+-- Applications (ForeignKey -> User, Service)
    |
    +-- Documents (ForeignKey -> Application)
        |
        +-- QueueTokens (ForeignKey -> Application, User)
```

Django ORM handles data migrations safely, preserving relationship constraints between user accounts, submitted requests, uploaded document files, and queue state objects.

5.9 Security Implementation

Security mechanisms incorporated within the application include:

- **Cross-Site Request Forgery (CSRF) Tokens:** Embedded in all POST forms to prevent unauthorized cross-domain submissions.
- **Password Encryption:** Credentials are stored using PBKDF2 algorithm with SHA256 hashes.
- **SQL Injection Prevention:** Parameterized ORM queries render standard SQL attacks ineffective.
- **Protected File Storage:** Uploaded applicant documents are kept in secure media directories accessible only via authenticated staff routes.

5.10 Testing of Implemented Features

Comprehensive testing was conducted throughout system development. Test cases and outcomes are summarized in Table 5.2.

Table 5.2: Functional Test Cases

Test Case	Expected Result	Status
Applicant Registration	Account created and assigned Applicant role.	Pass
User Login	Validates credentials and redirects to correct dashboard.	Pass
Unauthenticated Navigation	Redirects user to login view when accessing protected pages.	Pass
Application Submission	Saves application record and generates tracking reference.	Pass
Document Upload	Uploads file to media directory and links to application record.	Pass
Staff Verification Desk	Allows staff to view files and update processing status.	Pass
Queue Token Generation	Assigns daily token number sequentially.	Pass
Status Reflection	Status changes made by staff update on applicant dashboard instantly.	Pass

5.11 Summary

This chapter detailed the implementation phase of AkshayaAssist. The platform was successfully realized using Python, Django, HTML, CSS, JavaScript, Bootstrap, and relational database systems.

The implementation delivers robust user authentication, online service application forms, document upload management, staff verification portals, dynamic status tracking, and queue token generation. The modular architecture provides a strong foundation for integrating extended capabilities in future system iterations.

Bibliography

- [1] Django Software Foundation. *Django Documentation*.
<https://docs.djangoproject.com/>
- [2] Python Software Foundation. *Python Language Reference*.
<https://www.python.org/>
- [3] Bootstrap Team. *Bootstrap Framework Documentation*.
<https://getbootstrap.com/>
- [4] SQLite Development Team. *SQLite Database Documentation*.
<https://www.sqlite.org/>